Machine Learning Laboratory

Naive Bayes Classification: From Scratch to Bayes Optimal

Sentence role classification on the PubMed 20k RCT dataset

Name: [Your Name]
SRN: PES1UG24CS616
Course: Machine Learning, UE24CS352A (Semester V)
Department: Computer Science & Engineering, PES University
Date: 14 October 2026

1. Introduction

The task is **sentence role classification**: given one sentence from the abstract of a randomised controlled trial, predict which section it came from (BACKGROUND, OBJECTIVE, METHODS, RESULTS or CONCLUSIONS). Labelling sentences this way makes unstructured biomedical abstracts searchable by rhetorical role.

The purpose of the lab is to build a Multinomial Naive Bayes classifier from scratch in NumPy, verify it numerically against scikit-learn, and then study three things: how the smoothing strength alpha affects performance (Experiment 1), how the feature representation (counts versus TF-IDF, unigrams versus bigrams) interacts with alpha (Experiment 2), and whether combining several models weighted by their posterior probability, an approximation of the Bayes Optimal Classifier, beats its individual members (Part C). Macro-averaged F1 is the primary metric throughout, because the classes are unbalanced; accuracy is reported alongside it.

2. Dataset description

The data is the PubMed 20k RCT corpus, which ships pre-split. Each line is a (label, sentence) pair.

Split	Sentences	Purpose
Train	180,040	Fits the model (word counts and priors)
Dev	30,212	Chooses hyperparameters (alpha, n-gram range)
Test	30,135	Scored only after every choice is fixed

Class distribution (training set). The five classes are far from balanced. METHODS and RESULTS together make up about 65% of the data, and always predicting METHODS would already give 0.330 accuracy, which is why accuracy alone is a weak metric here.

Class	BACKGROUND	CONCLUSIONS	METHODS	OBJECTIVE	RESULTS
Share of train	12.1%	15.1%	33.0%	7.7%	32.2%

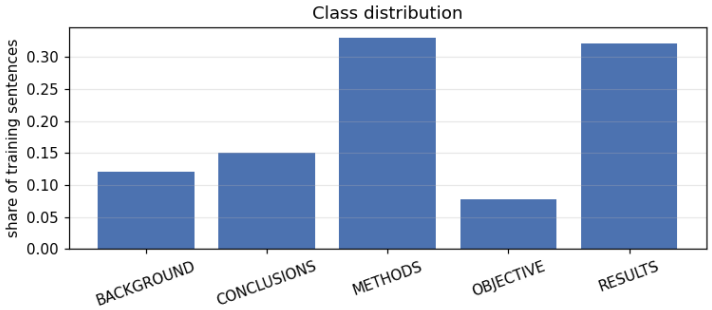


Figure 1. Class distribution of the training sentences.

Part C subsample. The last three digits of my SRN (PES1UG24CS616) are 616, so the Part C subsample has 10,000 + 616 = **10,616** training sentences (the first 10,616 of the training file). It was split stratified into 7,962 fitting and 2,654 held-out sentences (75/25). Its class mix is close to the full training set (METHODS 32.9%, RESULTS 31.4%, CONCLUSIONS 15.5%, BACKGROUND 12.2%, OBJECTIVE 7.9%).

Vocabulary size. Count features use lowercasing, accent stripping, English stop-word removal and min_df=2, fitted on the training split only, giving **V = 36,756** unigram features (the unigram TF-IDF vectoriser has the same vocabulary). Adding bigrams (1,2) grows it to 301,234 features. The Part C vectoriser (min_df=5) has 4,498 features on the

10,616-sentence subsample.

3. Methodology

Model. Naive Bayes picks the class with the highest posterior. Dropping the evidence term, which is the same for every class, and assuming the words are conditionally independent given the class:

$$\hat{c} = \operatorname{argmax}_c \, P(c) \prod_i P(w_i \mid c)^{x_i}$$

where x_i is the count of word i in the sentence. To avoid floating-point underflow everything is kept in log space, so scoring a whole dataset is one matrix multiplication:

$$\text{score}(c) = \log P(c) + \Sigma_i \, x_i \log P(w_i \mid c) \text{ , i.e. } X \cdot \text{feature_log_prob_}^{T} + \text{class_log_prior_}$$

Parameters and smoothing. The prior is the share of training *sentences* in the class, $P(c) = N_c/N$. The word likelihood uses additive (Laplace/Lidstone) smoothing:

$$P(w_i \mid c) = (\text{count}(w_i, c) + \alpha) / (\Sigma_j \, \text{count}(w_j, c) + \alpha V)$$

The αV term in the denominator keeps each class's word distribution summing to one, and the pseudo-count α guarantees that a single unseen word cannot drive a class's probability to zero. Baseline α = 1.

Verification. Before looking at any accuracy, the learned parameters were compared with scikit-learn's MultinomialNB on the same count matrix. The maximum absolute difference was **1.55e-15** for the log priors and **1.78e-15** for the log likelihoods (the requirement is below 1e-10), and the two implementations agree on 100.0000% of test predictions. This tests the numbers themselves, which accuracy cannot: for example, omitting the αV term lowers test accuracy by only one point (0.7357 to 0.7257), which would look like a modelling choice.

Protocol. Train fits the models, dev picks α and the n-gram range, and the test set is scored once per final configuration. Experiment 1 sweeps α over [0.01, 0.1, 0.5, 1.0, 2.0] on dev by macro-F1. Experiment 2 compares a default TF-IDF + MultinomialNB pipeline against one tuned by GridSearchCV (3-fold on dev, macro-F1) over n-gram range {(1,1), (1,2)} and α {0.1, 0.5, 1.0}, then refits the winner on the full training set.

Part C (approximate Bayes Optimal Classifier). Four hypotheses (Naive Bayes, Logistic Regression, Decision Tree, Random Forest) are fitted on the 7,962 fitting sentences. Each model's mean log-likelihood of the true labels on the 2,654 held-out sentences (probabilities clipped at 1e-12) is turned into a weight by a softmax with the maximum subtracted. A soft-voting ensemble with those weights is fitted on the full subsample. The exact Bayesian weights, from the *summed* log-likelihood, are computed for comparison.

4. Results and analysis

4.1 Complete results table (test set)

Experiment	Accuracy	Macro-F1
TF-IDF + NB (tuned: ngram, alpha)	0.7525	0.6709
Custom NB (counts, alpha=1)	0.7357	0.6660
Custom NB (counts, tuned alpha=2.0)	0.7375	0.6632
Part C: LogisticRegression	0.7161	0.6285
BOC approx. (soft vote, summed-LL weights)	0.7161	0.6285
BOC approx. (soft vote, mean-LL weights)	0.7056	0.5984
TF-IDF + MultinomialNB (default)	0.7136	0.5911
Part C: NaiveBayes	0.6877	0.5610
Part C: RandomForest	0.5889	0.4124
Part C: DecisionTree	0.5131	0.3736

Table 1. Every experiment, sorted by macro-F1. Rows labelled Part C and BOC use the 10,616-sentence subsample and should be compared with each other, not with the full-data rows. Conclusions below are drawn from macro-F1.

4.2 Experiment 1: smoothing strength

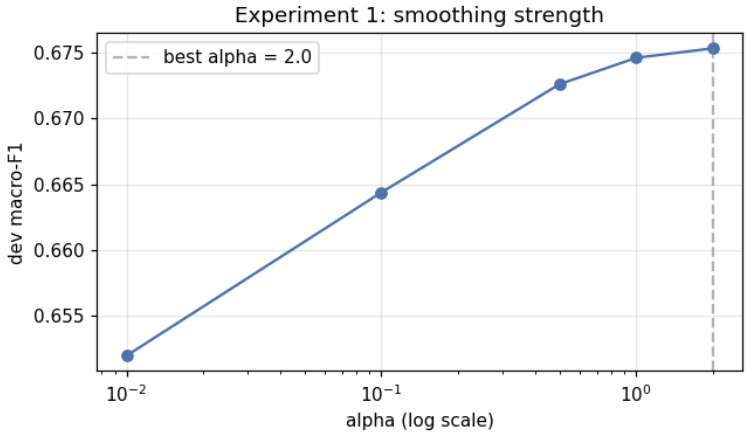


Figure 2. Dev macro-F1 against α (log scale). Dev scores: 0.01 → 0.6520, 0.1 → 0.6644, 0.5 → 0.6726, 1.0 → 0.6746, 2.0 → 0.6753.

Too little smoothing costs 0.023 macro-F1 on dev (α = 0.01 against α = 1), because a word unseen in a class is then about 100 times less probable than a word seen once and rare-word counts are mostly noise. From α = 0.5 upward the curve is nearly flat. The dev winner, α = 2.0 (at the edge of the grid), beats α = 1.0 by only 0.0007, and on test it scored 0.6632 against 0.6660 for α = 1: tuning around the optimum made no measurable difference.

4.3 Experiment 2: counts versus TF-IDF

The default TF-IDF model (unigrams, α = 1) scored 0.5911 macro-F1, well below the count model (0.6660). TF-IDF rows are L2-normalised, so a sentence carries about 3.3 units of mass instead of about 14 counts, and the smoothing mass αV = 36,756 equals 0.74 of OBJECTIVE's total class mass (0.19 for METHODS), against 0.17 and 0.04 with counts. The same α therefore flattens the class distributions far more, especially for the small classes. The grid search confirms it: for unigrams the CV macro-F1 falls from 0.6036 at α = 0.1 to 0.5597 at 0.5 and 0.5001 at 1.0. The best setting was bigrams with α = 0.1 (CV macro-F1 0.6235 against 0.6036 for unigrams). Refitted on the full training set it reached **0.6709 macro-F1 and 0.7525 accuracy** on test, a small gain (+0.005) over counts, bought with an 8x larger

vocabulary (301,234 features) and a slower fit.

4.4 Confusion matrices and which classes are confused

Per-class F1 (test)	BACKGROUND	CONCLUSIONS	METHODS	OBJECTIVE	RESULTS
Baseline NB (counts, α=1)	0.54	0.63	0.83	0.50	0.82
Tuned TF-IDF + NB	0.56	0.65	0.84	0.47	0.84

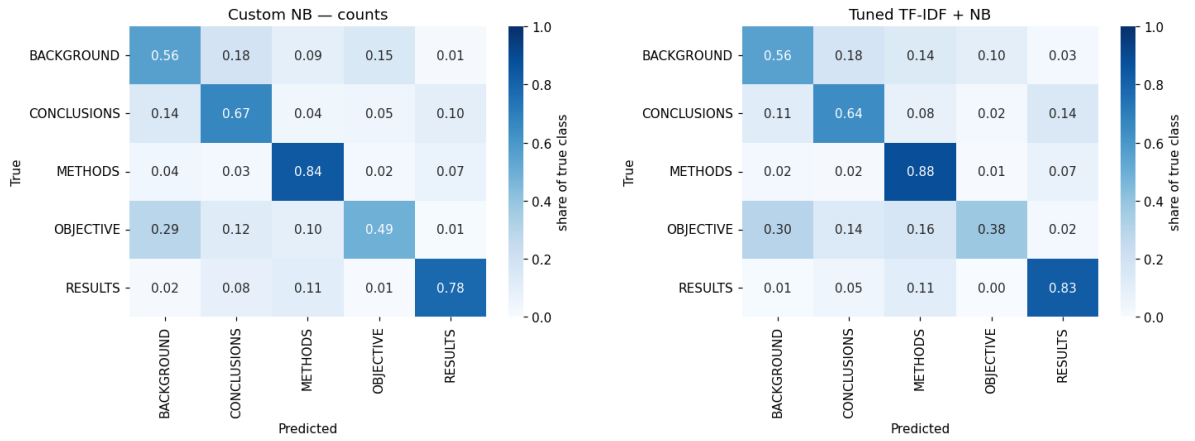


Figure 3. Row-normalised confusion matrices on the test set: baseline (left) and tuned TF-IDF + NB (right).

In both models the same blocks are confused. The largest off-diagonal entry is **OBJECTIVE predicted as BACKGROUND** (0.29 in the baseline, 0.30 tuned), followed by **BACKGROUND predicted as CONCLUSIONS** (0.18 in both) and BACKGROUND predicted as OBJECTIVE (0.15 baseline). METHODS and RESULTS are the best-separated classes (recall 0.84/0.78 baseline, 0.88/0.83 tuned), with their remaining errors mostly going to each other (RESULTS to METHODS: 0.11 in both). This is overlapping vocabulary rather than a generally weak model: objective and background sentences both name the condition, the patients and the treatment, and background and conclusion sentences are both general statements without numbers, so the few words that separate them are outvoted by the many they share. OBJECTIVE is the weakest class (baseline F1 0.50). The tuned model became more cautious about it: precision rose from 0.52 to 0.61 but recall fell from 0.49 to 0.38, which is why its OBJECTIVE F1 (0.47) is slightly lower even though overall macro-F1 improved.

4.5 Part C: posterior-weighted combination

Hypothesis	Mean log-lik	Held-out acc.	Weight (mean-LL)	Weight (summed-LL)	Test macro-F1
Naive Bayes	-0.9065	0.6507	0.3254	5.2e-45	0.5610
Logistic Regression	-0.8681	0.6839	0.3382	1.0000	0.6285
Decision Tree	-2.2598	0.5023	0.0841	0	0.3736
Random Forest	-1.1607	0.5780	0.2524	0	0.4124
BOC (mean-LL weights)					**0.5984**
BOC (summed-LL weights)					0.6285

Table 2. Posterior weights and test performance of the four hypotheses and the two ensembles.

The ensemble with mean-log-likelihood weights (macro-F1 0.5984) beats Naive Bayes, Random Forest and Decision Tree but is **0.030 below Logistic Regression alone** (0.6285). The mean-based weights are close to equal because the models' mean log-likelihoods differ by only a few tenths, so a Random Forest scoring 0.41 alone still carries a quarter of the vote.

The exact weights use the summed log-likelihood, which is the mean multiplied by n = 2,654; the 0.04 per-sentence lead of Logistic Regression over Naive Bayes becomes 102 log-units, so the exact posterior puts weight 1.000 on Logistic Regression and the ensemble reduces to that single model (0.6285). With only four hypotheses and thousands of held-out examples the Bayes Optimal approximation therefore collapses into model selection.

5. Conclusion

Smoothing. A weak α is harmful (dev macro-F1 0.6520 at α = 0.01 against 0.6746 at α = 1), but beyond α ≈ 0.5 the curve is flat; tuning α between 1 and 2 changed test macro-F1 by -0.003, which is noise. **Feature representation.** TF-IDF is not a free upgrade for a multinomial model: untuned it lost 0.075 macro-F1 to raw counts because its fractional, normalised values make a given α far stronger. With α and the n-gram range tuned together (bigrams, α = 0.1) it reached the best result of the lab, macro-F1 0.6709 and accuracy 0.7525, only +0.005 above plain count-based Naive Bayes. The implementation was verified against scikit-learn to about 1e-15 before any accuracy was measured. **Ensemble.** The posterior-weighted ensemble did **not** improve on its best member: 0.5984 against 0.6285 for Logistic Regression. The exact summed-likelihood weights collapse onto that one model, so the Bayes Optimal approximation brings no benefit from averaging when the hypothesis set is this small. All errors concentrate in the OBJECTIVE/BACKGROUND/CONCLUSIONS group, whose sentences share vocabulary, and all conclusions above are drawn from macro-F1.